清華大學
Tsinghua University

2026秋季

计算机系统概论

Introduction to Computer Systems

整 数

Zhang，Youhui（张悠慧）
zyh02@tsinghua.edu.cn









计算机体系结构

```c
int array[4] = {1, 2, 3, 4};
static int brray[4] = {1, 2, 3, 4};

static int intra_sum (int x[4], int y)
{
   return x[y-1];
}
int main()
{
    int val = intra_sum(array, 3) + inter_sum(brray, 3);
    return val;
}
```

main.c

编译

```
intra_sum:
        movslq  %esi, %rsi
        movl    -4(%rdi,%rsi,4), %eax
        ret
main:
        pushq   %rbx
        movl    $3, %esi
        movl    $array, %edi
        call    intra_sum
        movl    %eax, %ebx// val -> ebx
        movl    $3, %esi
        movl    $brray, %edi
        movl    $0, %eax
        call    inter_sum
        addl    %ebx, %eax// val -> eax
        popq    %rbx
        ret

brray:
        .long   1
        .long   2
        .long   3
        .long   4
array:
        .long   1
        .long   2
        .long   3
        .long   4
```

汇编指令

main.o

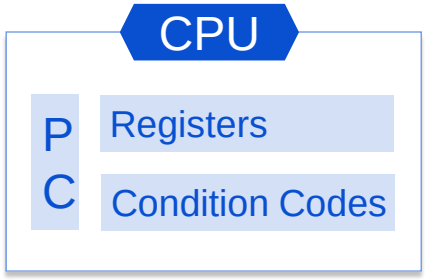
链接

```
00000000004004de <main>:
  4004de:       53                      push   %rbx
  4004df:       be 03 00 00 00          mov    $0x3,%esi
  4004e4:       bf 40 10 60 00          mov    $0x601040,%edi
  4004e9:       e8 e8 ff ff ff          callq  4004d6 <intra_sum>
  4004ee:       89 c3                   mov    %eax,%ebx
  4004f0:       be 03 00 00 00          mov    $0x3,%esi
  4004f5:       bf 30 10 60 00          mov    $0x601030,%edi
  4004fa:       b8 00 00 00 00          mov    $0x0,%eax
  4004ff:       e8 04 00 00 00          callq  400508 <inter_sum>
  400504:       01 d8                   add    %ebx,%eax
  400506:       5b                      pop    %rbx
  400507:       c3                      retq
```

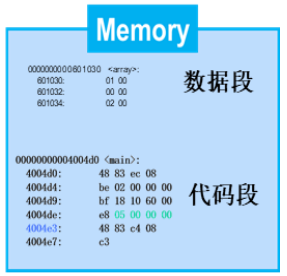
内存地址

仅给出部分内容

运行







C程序在硬件层面的表示

数据

- **整数 / 浮点数（第二讲）**
- **数组 / 结构（第五讲）**

代码

- ……

程序/数据在机器层面的表示与运行



目 录

CONTENTS

01 数的机器表示（初步）

02 整数的表示

预备知识

- **1K = 2^{10} = 1024（kilo）**
- **1M = 1024K = 2^{20}（Mega）**
- **1G = 1024M = 2^{30}（Giga）**
- **1T = 1024G = 2^{40}（Tera）**
- **1P = 1024T = 2^{50}（Peta）**
- **1E = 1024P = 2^{60}（Exa）**

- **1个二进制位：bit（比特）**
- **8个二进制位：Byte（字节） 1Byte = 8bit**
- **2个字节：Word（字） 1Word = 2Byte = 16bit***

*** x86架构下如此，RISC-V的话1-word = 32-bit**

数的机器表示

- **机器字(machine word)长**

计算机进行一次整数运算所能处理的二进制数据的位数

- 通常也指数据地址长度

32位字长

- 地址的表示空间是4GB
- 对很多内存需求量大的应用而言，非常有限

64位字长

- 地址的表示空间约是1.8 x 10^{19} bytes
- 目前的x86-64 机型实际支持 48位宽的地址: 256 TB

机器字在内存中的组织

地址按照字节(byte)来定位

- 机器字中**第一个字节**的地址
- **相邻机器字的地址**相差4 (32位字)
  或者8 (64位字)



字节序（Byte Ordering）

一个机器字内的各个字节如何排列?

- **大端（Big Endian）： Sun, PowerPC, 网络字节序, Java**
  最低权重字节（Least significant byte, LSB）占据高地址
- **小端（Little Endian）： x86, RISC-V, ARM(默认)**
  与LSB相反

数值是0x01234567，地址是0x100

	0x100	0x101	0x102	0x103			
Big Endian			01	23	45	67		
Little Endian			67	45	23	01		

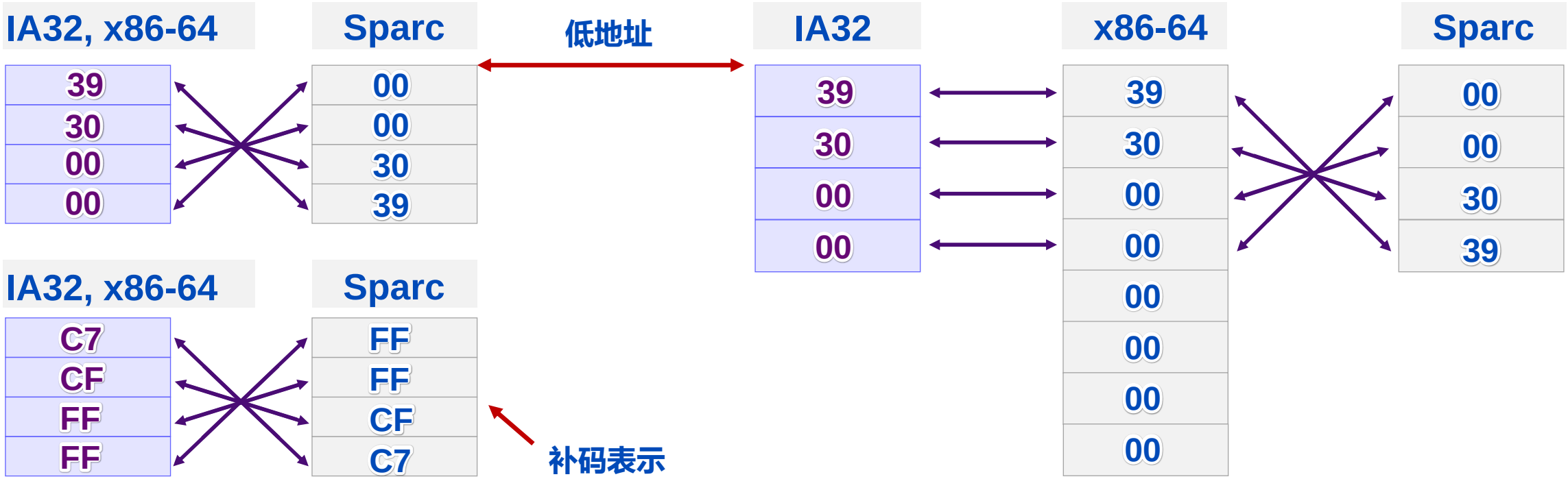


Gulliver's Travels

字节序（Byte Ordering）

```
int A = 12345 ;
int B = -12345 ;
long int C = 12345 ;
```

十进制: 12345
二进制: 0011 0000 0011 1001
十六进制: 3 0 3 9



网络字节序示例

发送端（x86小端）：

本地值 i = 0x12345678

内存布局：78 56 34 12（低地址→高地址）

htonl转换后：0x78563412（数值上的重排），

或等价：确保传输字节序为 12 34 56 78（大端）

发送字节流：[12][34][56][78]

接收端（ARM，被设为大端）：

接收字节流：[12][34][56][78]

ntohl：大端→本地（大端机器上无变化）= 0x12345678

本地内存布局：12 34 56 78（符合大端）

结果：i = 0x12345678 ✓

接收端（x86小端，若错误处理）：

直接memcpy：本地内存 12 34 56 78 （低→高地址）

小端解释：0x78563412 ✗ （数值颠倒）

思考：如果通过磁盘拷贝传递呢?

字节序是一个如何解释数据存储顺序的问题

Show Bytes示例

```
typedef unsigned char *byte_pointer;

void show_bytes(byte_pointer start, size_t len)
{
    for (int i = 0; i < len; i++)
    {
        printf("%.2x\t", start[i]);
    }
}

void show_int(int x)
{
    show_bytes((byte_pointer)&x, sizeof(int));
}

void show_double(double x)
…

void show_float(float x)
…

int main(void)
{
    int x = 0x12345678;
    show_int(x);
}
```

start[0]		start[1]		start[2]		start[3]	
byte0	byte1	byte2	byte3	byte4	byte5	byte6	byte7
8	7	6	5	4	3	2	1

LSB MSB (most significant bit)

```
zhang@ubuntu:~/work$ ./show
78      56      34      12
zhang@ubuntu:~/work$ !vi
```

整数表示

C语言中基本数据类型的大小 (单位为字节)

C Data Type	32-bit	x86-32	x86-64	
char	1	1	1	
short	2	2	2	
int	4	4	4	
long	4	4	8	(Linux)
long long	8	8	8	
float	4	4	4	
double	8	8	8	
long double	8	10/12	10/16	
char * or any other pointer	4	4	8	
	/	/	/	

计算机中整数的二进制编码方式（w表示字长）

补码让硬件更简洁：加法和减法在电路层面统一

- **无符号数（原码表示）**

$$B2U(X) = \sum_{i=0}^{w-1} x_i \cdot 2^i$$

- **带符号数（补码，Two's Complement）**

$$B2T(X) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$$

符号位

```
short int x =  12345 ;
short int y = - 12345 ;
```

	Decimal	Hex	Binary
x	12345	30 39	00110000 00111001
y	-12345	CF C7	11001111 11000111

符号位（sign bit）

- **对于补码表示, MSB （Most Significant Bit）表示整数的符号**
  - 1 for negative
  - 0 for nonnegative

- **非负数：补码 = 原码**（反码是原码各位取反）
- **负数：补码 = 绝对值的反码 + 1 = 2^w + 该负数**
- **一个数的补码：它的相反数的补码的按位取反加一**

计算机中整数的二进制编码方式（w表示字长）

补码让硬件更简洁：加法和减法在电路层面统一

- **无符号数（原码表示）**

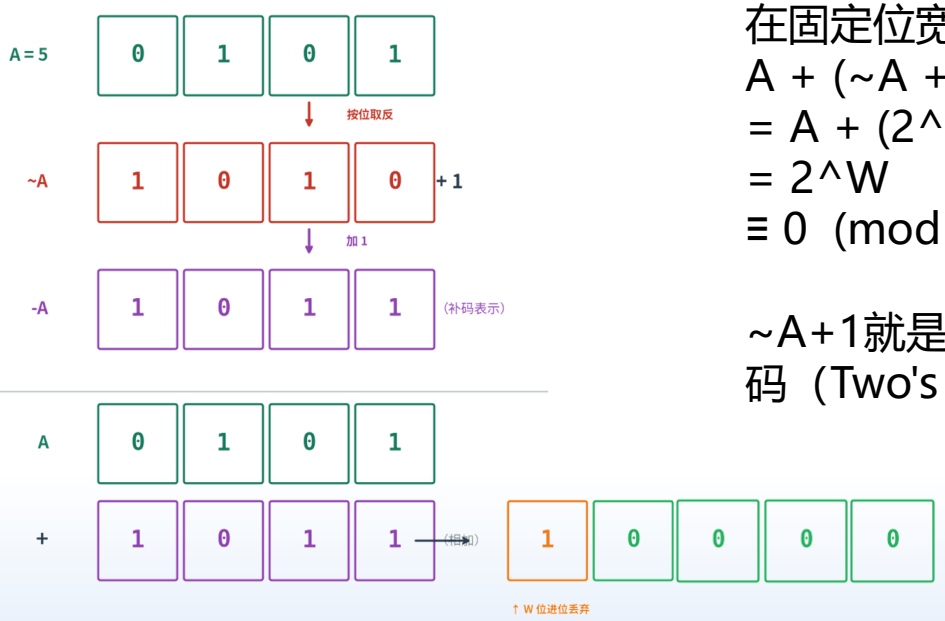
$$B2U(X) = \sum_{i=0}^{w-1} x_i \cdot 2^i$$

- **带符号数（补码，Two's Complement）**

$$B2T(X) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$$

符号位

补码为何是这种表示方式，或者说带符号数为何如此表示?



在固定位宽W的系统中，所有运算均是**模 2^W** 的

A + (~A + 1)
= A + (2^W - A)
= 2^W
≡ 0 (mod 2^W)

~A+1就是**A在模2^W下的加法逆元**，这也正是补码（Two's Complement）这个名字的来源

取值范围

- 无符号数
  - $U_{min} = 0$ 000…0
  - $U_{max} = 2^w - 1$ 111…1
- 带符号数
  - $T_{min} = -2^{w-1}$ 100…0
  - $T_{max} = 2^{w-1} - 1$ 011…1
- 负1 = 111…1

假设字长为16(w=16)

	Decimal	Hex	Binary
U_{max}	65535	FF FF	11111111 11111111
T_{max}	32767	7F FF	01111111 11111111
T_{min}	-32768	80 00	10000000 00000000
-1	-1	FF FF	11111111 11111111
0	0	00 00	00000000 00000000

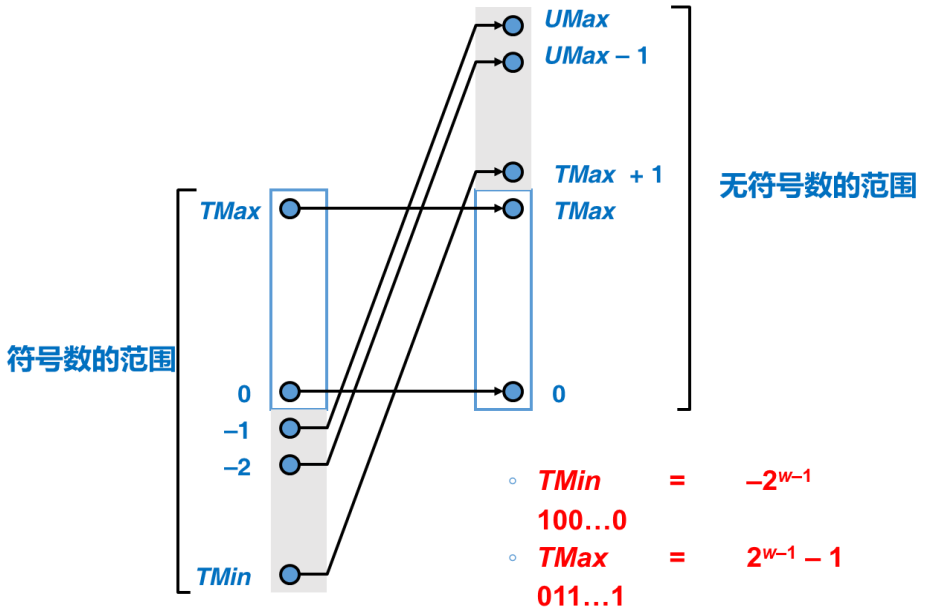
无符号数与带符号数

X	B2U(X)	B2T(X)
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	-8
1001	9	-7
1010	10	-6
1011	11	-5
1100	12	-4
1101	13	-3
1110	14	-2
1111	15	-1

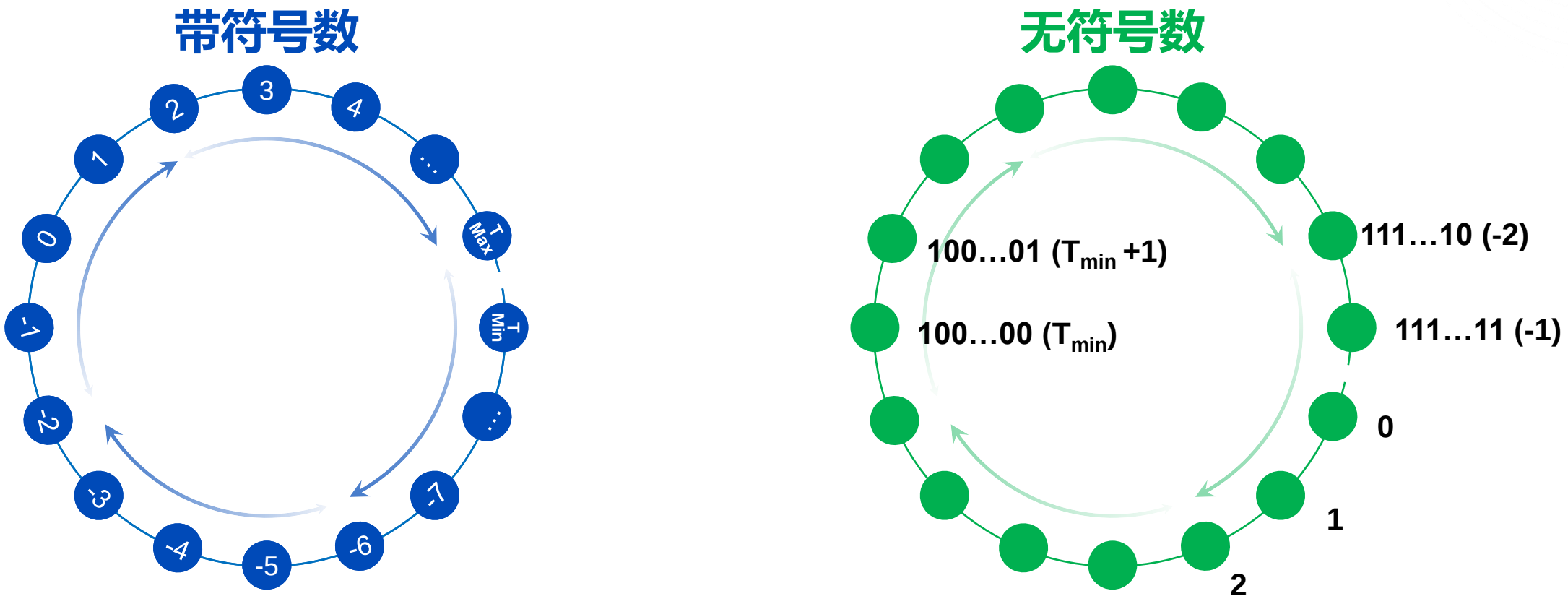
无符号数与带符号数之间的转换

- **二进制位串的表示是不变的**

$$ux = \begin{cases} x & x \geq 0 \\ x + 2^{w} & x < 0 \end{cases}$$



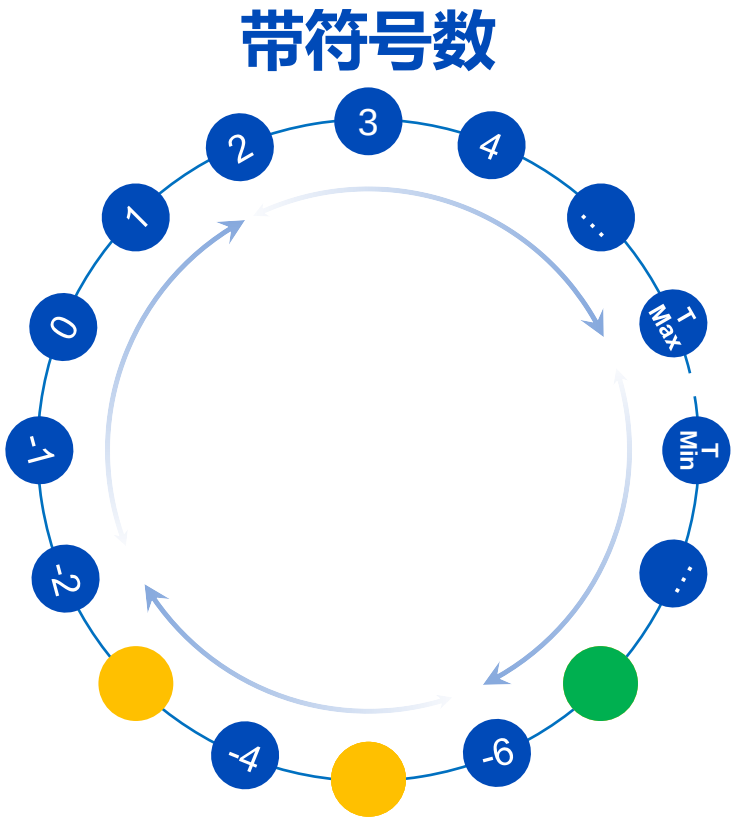
带符号/无符号整数的加减操作



带符号数的补码系统的核心：

把负整数映射到无符号数的高数值范围

带符号/无符号整数的加减操作



x：圆环上的任意一个位子　**y**：正数（y>0）

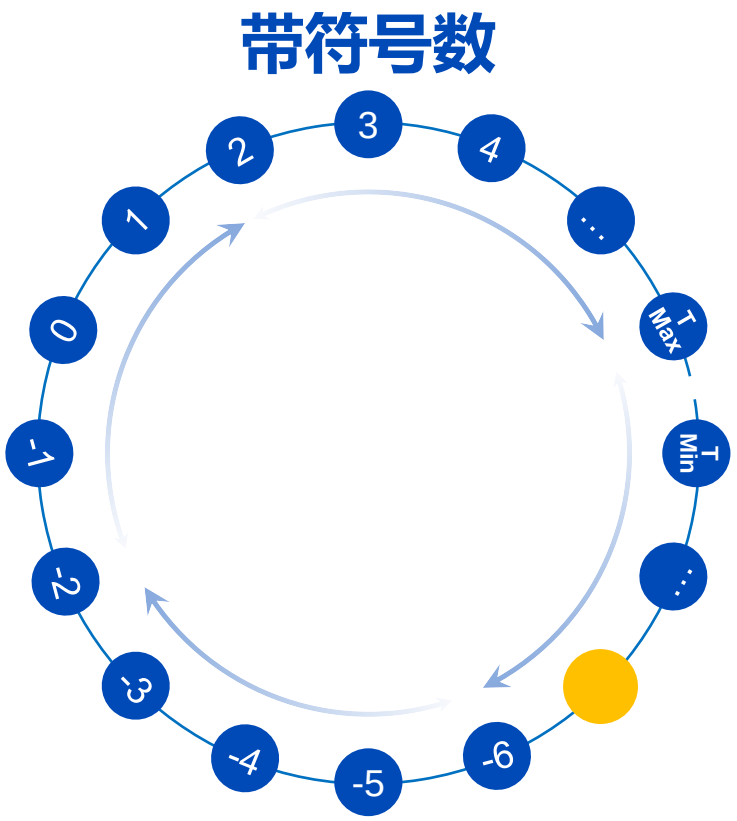
x+y： x顺时针移动y个位子

x-y ： x逆时针移动y个位子

x逆时针移动y个位子=x顺时针移动（R-y）个位子

w位整数：R=2^w

带符号/无符号整数的加减操作



减法可以用取反和加法代替

实现了加法和减法在电路层面的统一

公式1

x - y = x逆时针移动y个位子

= 顺时针移动【R-y】个位子

公式2

R - 1 = y + (~y) =全1的w位二进制串

R - y = (~y) + 1

公式3

x – y = x + (R-y)

- y = R-y

- y = (~y) + 1

x - y = x + (~y) + 1

补码加法公式*

$[x]补+[y]补 \equiv [x+y]补 \ (\mathrm{mod}\ 2^w)$

意义：

负整数用补码表示后，可以和正整数一样来处理，这样（处理器的）运算器只需要加法器

证明：

$[y]补= 2^w+y \ (y<0)$

- x>0;y>0 由于正数的补码和原码一致，x+y>0，所以在这种情况下[x]补 + [y]补 ≡ [x+y]补
- x>0;y<0且x+y>0 我们有如下的等式：[x]补=x [y]补= 2^w+y，所以，[x]补+ [y]补 =x+y+2^w ≡ x+y= [x+y]补:
- x>0;y<0且x+y<0，可以发现以下等式：[x]补=x [y]补= 2^w+y，所以，[x]补+ [y]补 =x+y+2^w = [x+y]补:

C语言中的无符号数与带符号数

常数（Constants）

- **默认是带符号数**
  - 如果有“U” 作为后缀则是无符号数，如 0U, 4294967259U

无符号数与带符号数混合使用

- **带符号数默认转换为无符号数**
  - 包括比较操作符　　实例（w=32）

$Constant_1$	$Constant_2$	比较大小
0	0U	
-1	0	
-1	0U	
2147483647	-2147483647-1	
2147483647U	-2147483647-1	
-1	-2	
(unsigned)-1	-2	
2147483647	2147483648U	
2147483647	(int)2147483648U	

C语言中的无符号数与带符号数

$Constant_1$	$Constant_2$	比较大小	带符号数或无符号数处理
0	0U	==	unsigned
-1	0	<	signed
-1	0U	>	unsigned
2147483647	-2147483647-1	>	signed
2147483647U	-2147483647-1	<	unsigned
-1	-2	>	signed
（unsigned）-1	-2	>	unsigned
2147483647	2147483648U	<	unsigned
2147483647	（int）2147483648U	>	signed

何时采用无符号数

- 模运算
- 按位运算
- 建议：不能仅仅因为取值范围是非负而使用

示例一

```
unsigned i;
for (i = cnt-2; i >= 0; i--)
    a[i] += a[i+1];
```

示例二

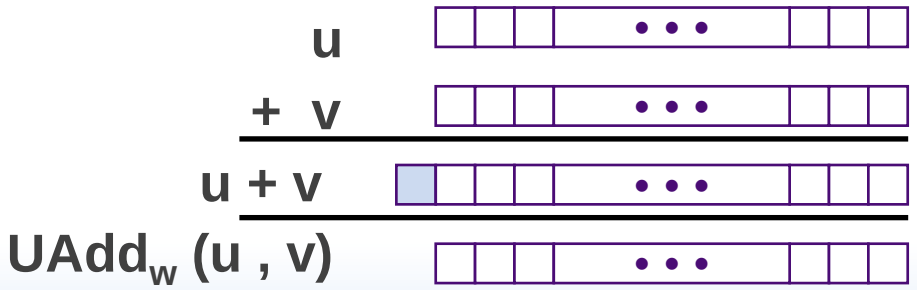
```
#define DELTA sizeof(int)
int i;
for (i = CNT; i-DELTA >= 0; i-= DELTA)
…
```

无符号数加法

操作数位宽: w bits

真实结果位宽: w + 1 bits

放弃进位: w bits



$$s = \mathrm{UAdd}_w(u, v) = (u + v) \bmod 2^w$$

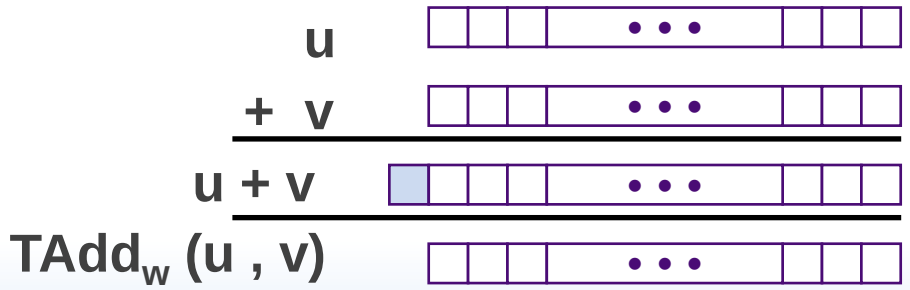
$$\mathrm{UAdd}_w(u,v) = \begin{cases} u + v & u + v < 2^w \\ u + v - 2^w & u + v \geq 2^w \quad \text{溢出} \end{cases}$$

补码加法

操作数位宽: w bits

真实结果位宽: w + 1 bits

放弃进位: w bits



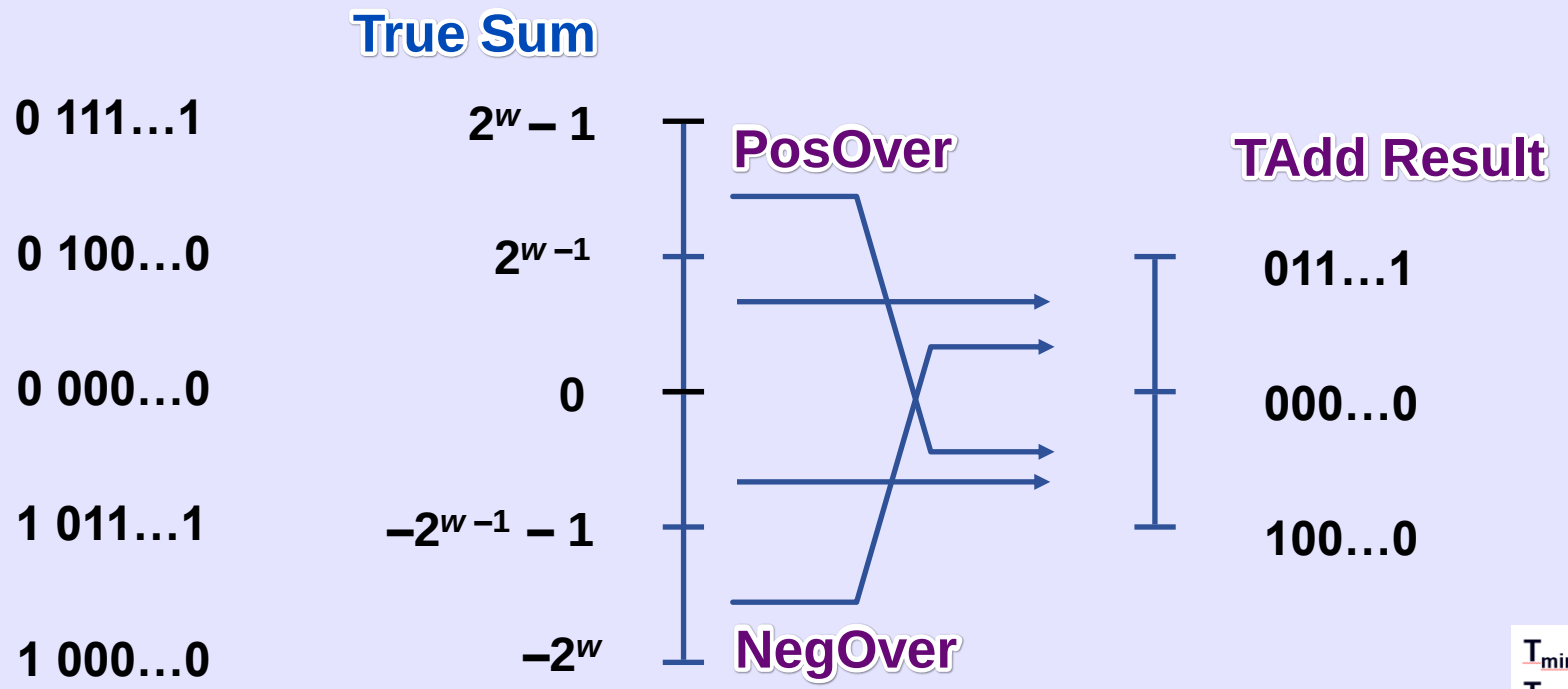
与无符号数的一致

- C语言中的带符号数 / 无符号数加法

```
int s, t, u, v;
s = (int) ((unsigned) u + (unsigned) v);
t = u + v
```

- s == t

补码加法的溢出



$$TAdd_w(u,v) = \begin{cases} u + v + 2^w & u + v < TMin_w \\ u + v & TMin_w \le u + v \le TMax_w \\ u + v - 2^w & TMax_w < u + v \end{cases}$$

NegOver / 向下溢出

PosOver / 向上溢出

填空题

- **已知某32位整数X，其值为-101（十进制）**
  则其16进制补码为 [填空1]

- **另一32位整数Y的补码为0xFFFFFF6A**
  则 X + Y 的16进制补码（32位）为 [填空2]
  X – Y 的16进制补码为 [填空3]

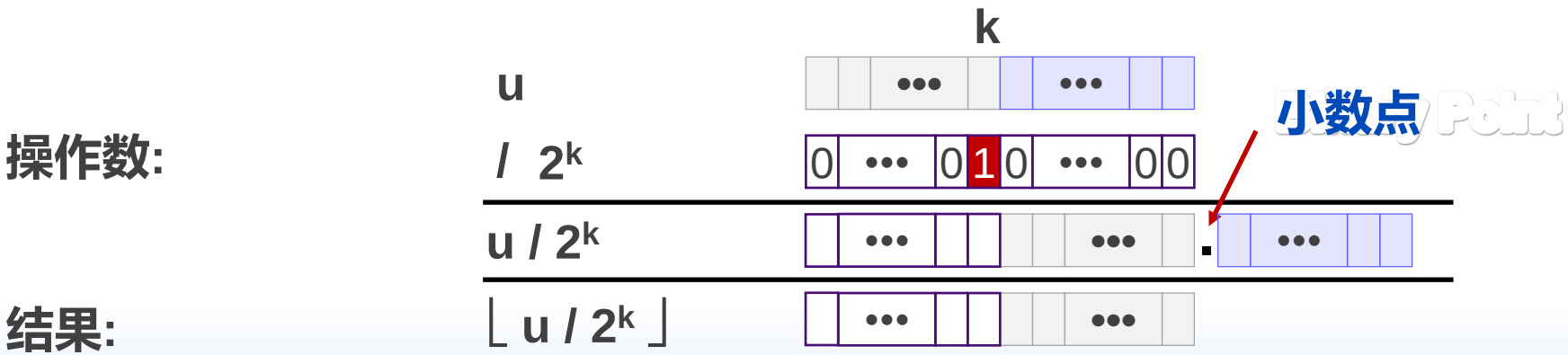
思考一个问题（3.2讲解决）

- 已知1：补码加法在位模式表示层面与无符号二进制加法完全一致
- 已知2：x86处理器使用同一条addl指令来进行无符号/带符号整数加法，不加区分（以32位整数为例）

问题：C语言层面的无符号/带符号的语义区分如何完成?

无符号整数除以2的k次幂

u >> k 等价于 $\lfloor u / 2^k \rfloor$　　**采用逻辑右移**



	Division	Computed	Hex	Binary
x	15213	15213	3B 6D	00111011 01101101
x >> 1	7606.5	7606	1D B6	00011101 10110110
x >> 4	950.8125	950	03 B6	00000011 10110110
x >> 8	59.4257813	59	00 3B	00000000 00111011

无符号整数除以2的k次幂

```
unsigned udiv8(unsigned x)
{
   return x/8;
}
```

等价的汇编代码

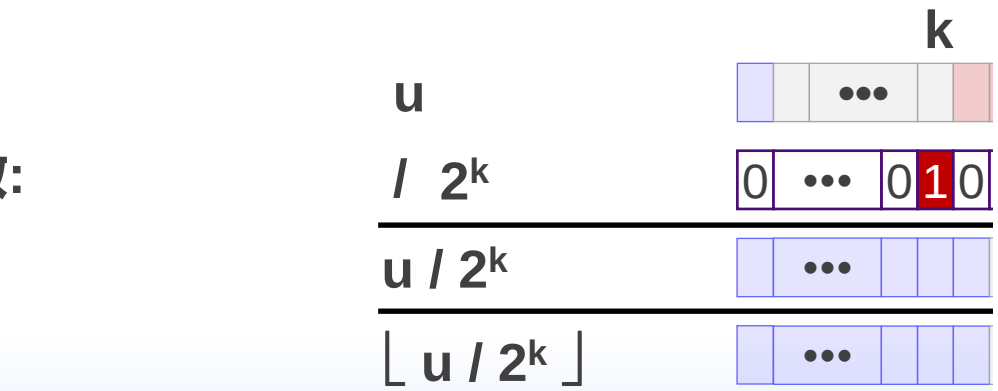
```
shrl      $3, %eax
```

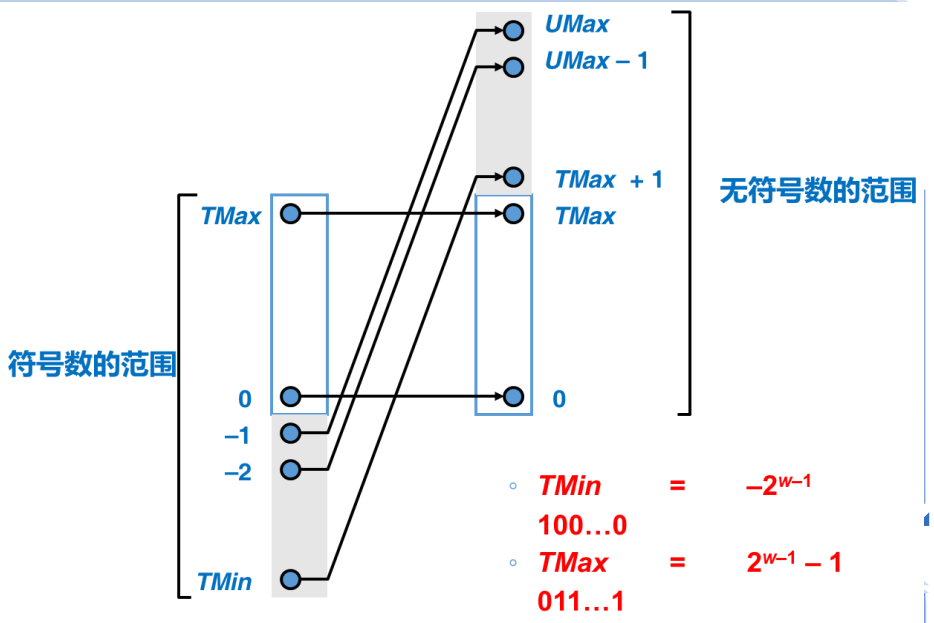
C代码的解释

```
# 逻辑右移
  return x >> 3;
```

带符号整数除以2的k次幂

x >> k 等价于 ⌊ x / 2^k ⌋　　采用算术右移　　但是x < 0时，舍入错误





	Division	Computed	Hex	Binary
x	-15213	-15213	C4 93	11000100 10010011
x >> 1	-7606.5	-7607	E2 49	11100010 01001001
x >> 4	-950.8125	-951	FC 49	11111100 01001001
x >> 8	-59.4257813	-60	FF C4	11111111 11000100

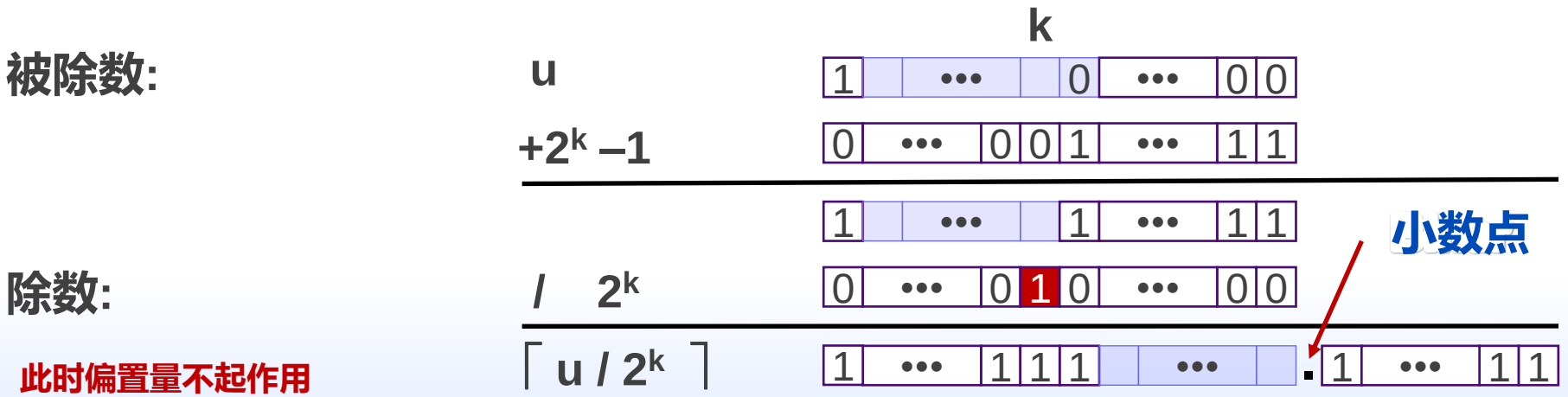
带符号整数除以2的k次幂

- 期望的结果是$\lceil x / 2^k \rceil$ (需要向0舍入，而不是向”下”舍入)
- 所以引入偏置量$\lfloor (x+2^k-1) / 2^k \rfloor$

Bias / 偏置量

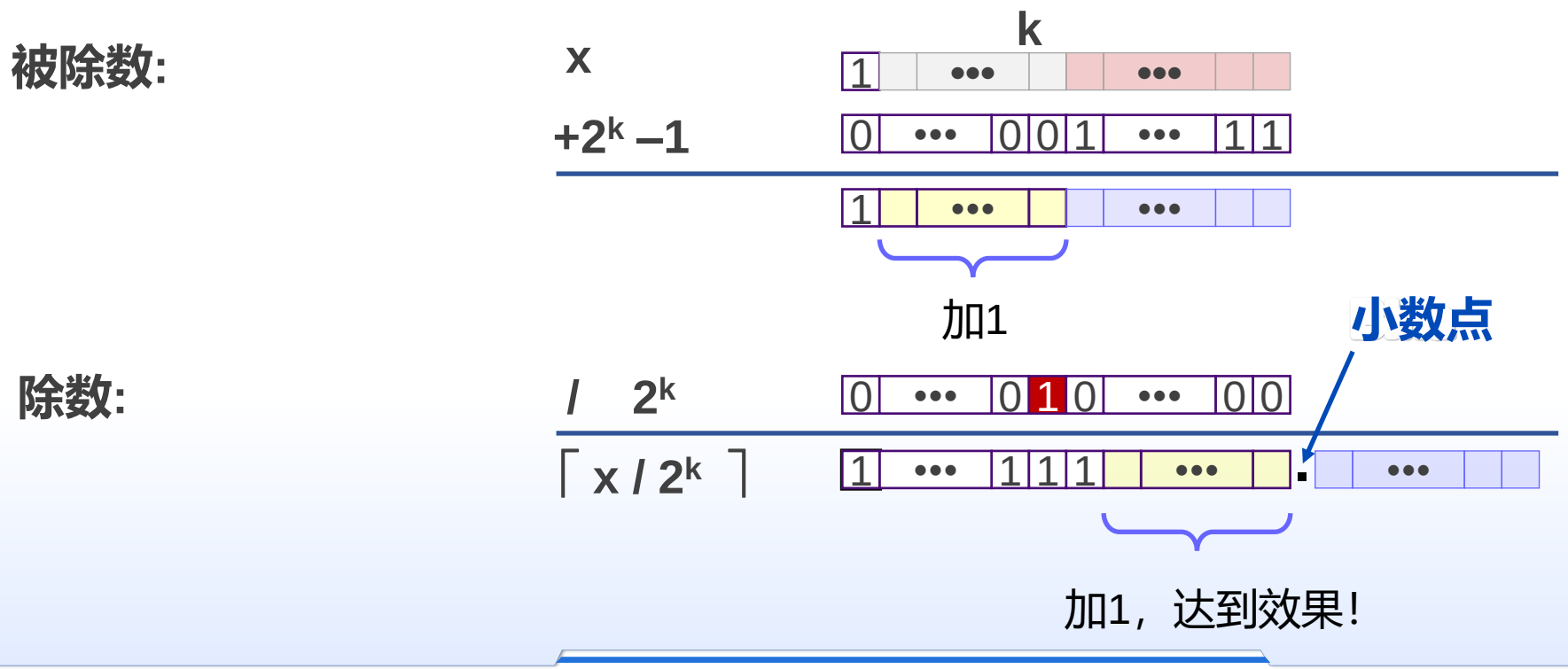
  - C语言： (x + (1<<k)-1) >> k

情况一: 能够被2^k整除



带符号整数除以2的k次幂

情况二


被除数:
x
k
1 ••• ••• •••

带符号整数除以2的k次幂

C函数

```
int idiv8(int x)
{
    return x/8;
}
```

等价的汇编代码

```
testl   %eax, %eax
  js    L4
L3:
  sarl  $3, %eax
  ret
L4:
  addl  $7, %eax
  jmp   L3
```

C代码的解释

```
if x < 0
        x += 7;
# 算术右移
return x >> 3;
```

C语言中的无符号数与带符号数

$Constant_1$	$Constant_2$	比较大小	带符号数或无符号数处理
0	0U	==	unsigned
-1	0	<	signed
-1	0U	>	unsigned
2147483647	-2147483647-1	>	signed
2147483647U	-2147483647-1	<	unsigned
-1	-2	>	signed
（unsigned）-1	-2	>	unsigned
2147483647	2147483648U	<	unsigned
2147483647	（int）2147483648U	>	signed

整数运算的一些可能极端情况

判断以下的推断或者关系式是否成立 (不成立则给出示例)

- x，y 为32位带符号整数
- ux，uy为与x，y具有相同二进制表示的32位无符号整数

- x < 0 ⇒ ((x*2) < 0)
- ux >=0
- x & 7 == 7 ⇒ ((x<<30) < 0
- ux >-1
- x > y ⇒ -x < -y
- x > 0 && y > 0 ⇒ x + y > 0

- x >= 0 ⇒ -x <= 0
- x <= 0 ⇒ -x >= 0
- (x|-x) >> 31 == -1
- ux >> 3 == ux/8
- x >> 3 == x/8
- x & (x-1) != 0

以上仅从**带符号整数、无符号数的定义出发**，不涉及C语言的编译器具体实现，有兴趣同学可以深入参考C语言的“未定义行为”